

Walkthrough: Run tests and view code coverage

0 out of 3 rated this helpful

[This documentation is for preview only, and is subject to change in later releases. Blank topics are included as placeholders.]

To see what proportion of your project's code is actually being tested, use the code coverage feature of Visual Studio 2012 RC. To do this, first edit the run configuration to indicate the assembly containing the code whose coverage you want to measure; then, run tests on that code. Detailed code coverage statistics appear in a window, and you can also see, line-by-line, which code has been tested.

Prerequisites

- Visual Studio Premium 2010 or Visual Studio Ultimate 2010
- Perform the steps in the procedure "Run a Unit Test and Fix Your Code" in [Walkthrough: Creating and running unit tests for managed code](#). This creates the two tests that you will run in the following procedure.

Run Tests and View Code Coverage

To run tests and view code coverage

1. In Solution Explorer, note the name of your solution. If you used the project from [Walkthrough: Creating and running unit tests for managed code](#), the solution's name is Bank. This solution contains the code-under-test.
2. In Solution Explorer, under Solution Items, double-click the test settings file, Local.testsettings.

The **Test Settings** dialog box is displayed.
3. Select **Data and Diagnostics**.
4. Under **Role**, select **<Local machine only>** as the role to use to collect code coverage data.

 **Caution**

For code coverage data this must be the role that will run the tests.

5. To modify the default code coverage settings, in the list of data diagnostic adapters select the check box for **Code Coverage** and then click **Configure** located immediately above the list of data diagnostic adapters.

The **Code Coverage Detail** dialog box to configure code coverage collection is displayed.

 **Caution**

Collecting code coverage data does not work if you also have the test setting configured to collect IntelliTrace information.

6. From the list, select the artifacts that you want to instrument.
7. (Optional) To add another assembly that is not displayed, click **Add Assembly**.

The **Choose Assemblies to Instrument** dialog box is displayed.
 - a. Locate the assembly file (.exe, .dll, or .ocx) that you want to include in code coverage and then click **Open**. The file is added to the list.
8. (Optional) Select **Instrument assemblies in place** to instrument the files in the location where they are built or after you copy them to a deployment directory. For more information about where to instrument your assemblies, see [Choosing the Instrumentation Folder](#).
9. (Optional) If any one of your assemblies that you added have a strong name, you might have to re-sign these assemblies. Click (...) to locate the key file that must be used when they are re-signed. For more information about how assemblies are re-signed, see [Instrumenting and Re-Signing Assemblies](#).
10. Click **OK**. The code coverage settings are now configured and saved for your test settings.

 **Note**

To reset the configuration for this diagnostic data adapter, click **Reset to default configuration**.

11. Click **Save As** and then click **Save** in the dialog box. A message box appears, prompting you to save the existing file. Click **Yes** in the message box to replace the existing file.
12. On the **Test** menu, point to **Select Active Test Settings**. A submenu displays all the test settings in the solution. Put a check mark next to the test settings that you just edited, Local.testsettings. This makes it the active test settings.
13. In the **Test List Editor**, select the check boxes next to **CreditTest** and **DebitTest**, right-click, and then click **Run Checked Tests**.

The two tests run.
14. On the **Test Tools** toolbar, click **Code Coverage Results**.

The **Code Coverage Results** window opens.
15. In the **Code Coverage Results** window, the **Hierarchy** column displays one node that contains data for all the code coverage achieved in the latest test run. The test run node is

named using the format <user name>@<computer name> <date> <time>. Expand this node.

16. Expand the node for the assembly, Bank.dll, for the namespace, BankAccountNS, and for the BankAccount class.
17. The rows within the BankAccount class represent its methods. The columns in this table display coverage statistics for individual methods, for classes, and for the entire namespace.
18. Double-click the row for the **Debit** method.

The Class1.cs source-code file opens to the Debit method. In this file, you can see code highlighting. Lines highlighted light blue were exercised in the test run, lines highlighted beige were partially exercised and lines highlighted reddish brown were not exercised at all. By scrolling, you can see the coverage for the other methods in this file.

If you selected the check box for TestProject1.dll in step 7, you can open Class1Test.cs, the source-code file that contains your unit tests, to see which test methods were exercised. The same highlighting scheme applies: light blue indicates exercised code; beige indicates a partially exercised code path, and reddish brown indicates a code path that was untraveled in the test run.

See Also

Tasks

[Walkthrough: Creating and running unit tests for managed code](#)

[Sample project for creating unit tests](#)

[Create Test Settings to Run Automated Tests from Visual Studio](#)

[How to: Apply Test Settings from Microsoft Visual Studio](#)